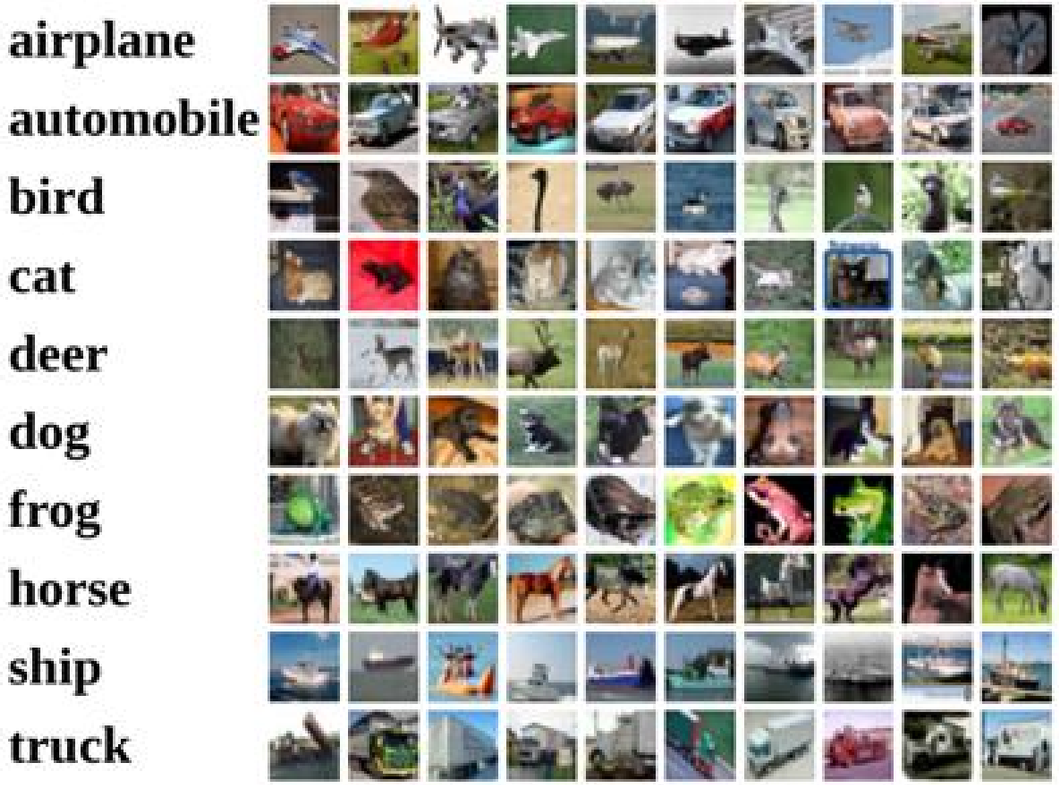
Image Classification

CS172 · Computer Vision I

Jiayuan Gu

ShanghaiTech University



Lecture overview

1. **The classification task**
   Images, labels, and appearance variation
2. **Nearest-neighbor classification**
   Similarity, voting, and the choice of distance
3. **Model selection**
   Generalization, validation, and final evaluation
4. **Linear classification**
   Class scores, image templates, and decision boundaries
5. **Probabilistic classification**
   Scores → probabilities → label likelihood → loss

An image, one category



Given an image and a fixed set of categories, predict one category.

$$\hat{y} = f(x) \in \{1, \ldots, C\}$$

dog	**cat**	truck	airplane

> The label describes the image as a whole.

Cat: Nikita · CC BY 2.0, via CS231n

Pixels and meaning



We see an object. The input is an array.

$$x \in \{0, \ldots, 255\}^{H \times W \times 3}$$

41	72	101	132
58	95	144	183
83	123	175	201
110	153	192	226

Illustrative values from one channel.

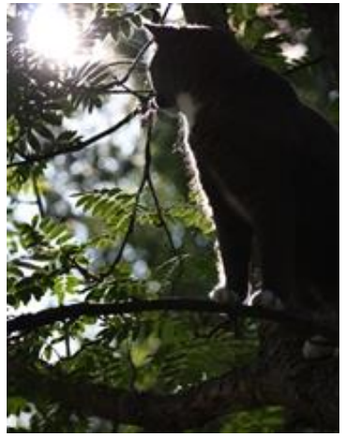
> Which patterns in this array support “cat”?

Cat: Nikita · CC BY 2.0, via CS231n

Why is image classification difficult?

One category can produce many different pixel arrays.



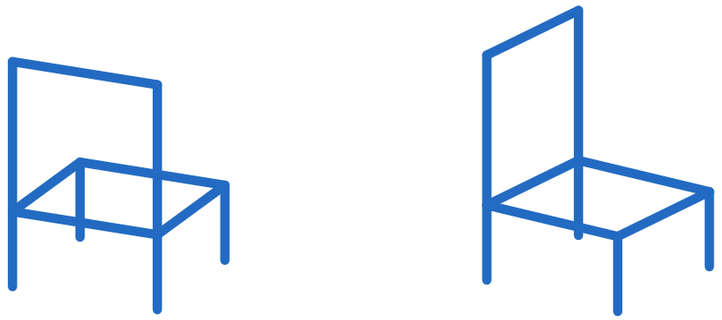




> A classifier should preserve the label when appearance or visibility changes.

Variation within one category

Viewpoint



View A View B

A new camera pose changes projected geometry and visibility.

Illumination



Lighting changes intensity, shadows, and color.

> The category can remain stable despite large changes in the image.

Clutter and occlusion



Background clutter

Many pixels belong to the surroundings.

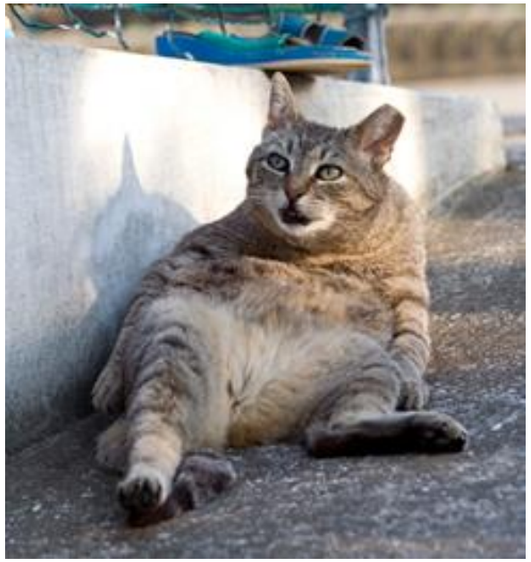


Occlusion

Only part of the object may be visible.

> Which evidence is useful, and which evidence is incidental?

Deformation and category diversity







Deformation

The same object can change its shape and pose.

Intraclass variation

Different instances share a label despite different appearances.

Context can help or mislead



Object appearance provides evidence.

Surrounding objects and scene layout also provide evidence.

> If every training cat sits on grass, what happens indoors?

Useful correlations may fail when the environment changes.

Photo: public domain, via CS231n

Representations for image classification

Classical approach

image → designed features → learned classifier

Examples: gradients, local descriptors, and feature histograms.

Deep learning approach

image → learned representation → class scores

The representation and classifier can be trained together.

> This lecture introduces nearest-neighbor and linear classifiers; both can use raw pixels or feature vectors.

Learning from labeled examples

Training

Labeled examples $\{(x_i, y_i)\}_{i=1}^{N}$

↓

Learning procedure

↓

Classifier

Prediction

New image x

↓

Classifier

↓

Predicted label $\hat{y}$

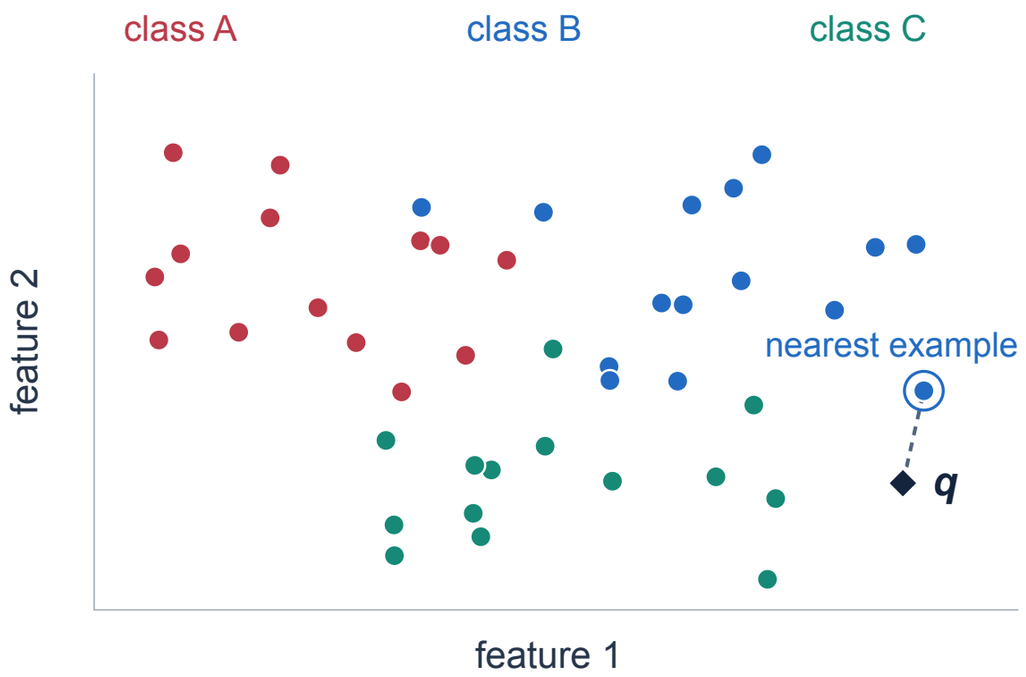
> The goal is accuracy on new examples from the intended setting.

Nearest-neighbor classification

Can similarity to labeled examples determine a category?

$$\text{query image} \quad \longleftrightarrow \quad \text{labeled examples}$$

The nearest-neighbor rule



The diamond marks the unlabeled query $\mathbf{q}$.

Training

Store N labeled examples:

$$\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$$

Prediction

For a query $\mathbf{q}$, find its closest stored example.

$$i^\star = \arg\min_{1 \le i \le N} d(\mathbf{q}, \mathbf{x}^{(i)})$$

$$\hat{y}(\mathbf{q}) = y^{(i^\star)}$$

A distance calculation

Flatten each image into a vector with $D = 4$ coordinates.

Query $\mathbf{q}$			**Stored example $\mathbf{x}^{(i)}$**	
10	20	versus	12	17
30	40		30	45

$$d_{\mathrm{L1}}(\mathbf{q}, \mathbf{x}^{(i)}) = \sum_{j=1}^{D} |q_j - x_j^{(i)}|$$

$$d_{\mathrm{L1}} = 2 + 3 + 0 + 5 = 10$$

$$d_{\mathrm{L2}} = \sqrt{(-2)^2 + 3^2 + 0^2 + (-5)^2} = \sqrt{38}$$

Superscript (i): example index. Subscript j: coordinate index.

Nearest neighbor in NumPy

```
# X_train: (N, D), q: (D,), y_train: (N,)
# Rows index examples; columns index coordinates.
X = X_train.astype(np.float32)
q = q.astype(np.float32)

distances = np.abs(X - q).sum(axis=1)
i_star = np.argmin(distances)
y_pred = y_train[i_star]
```

Row i stores $\mathbf{x}^{(i)}$; the code returns $\hat{y}(\mathbf{q})$.

Cast before subtraction to avoid uint8 wraparound. NumPy uses zero-based indices.

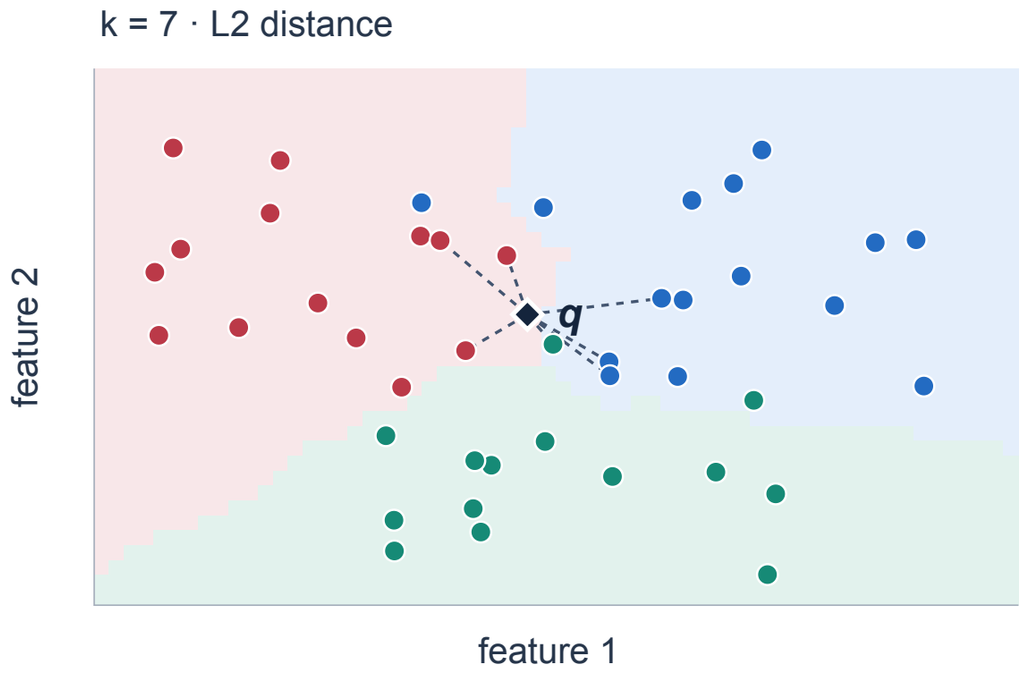
Where nearest neighbor spends time

N stored examples, each with D coordinates.

Operation or storage	Brute-force cost
Keep references to existing arrays	$O(1)$
Copy and store the training data	$O(ND)$
Predict one label $\hat{y}(\mathbf{q})$	$O(ND)$
Memory for the stored data	$O(ND)$

> For fixed D, direct prediction grows linearly with N.

Voting among nearby examples



$\mathcal{I}_k(\mathbf{q})$ contains the indices of the k nearest stored examples.

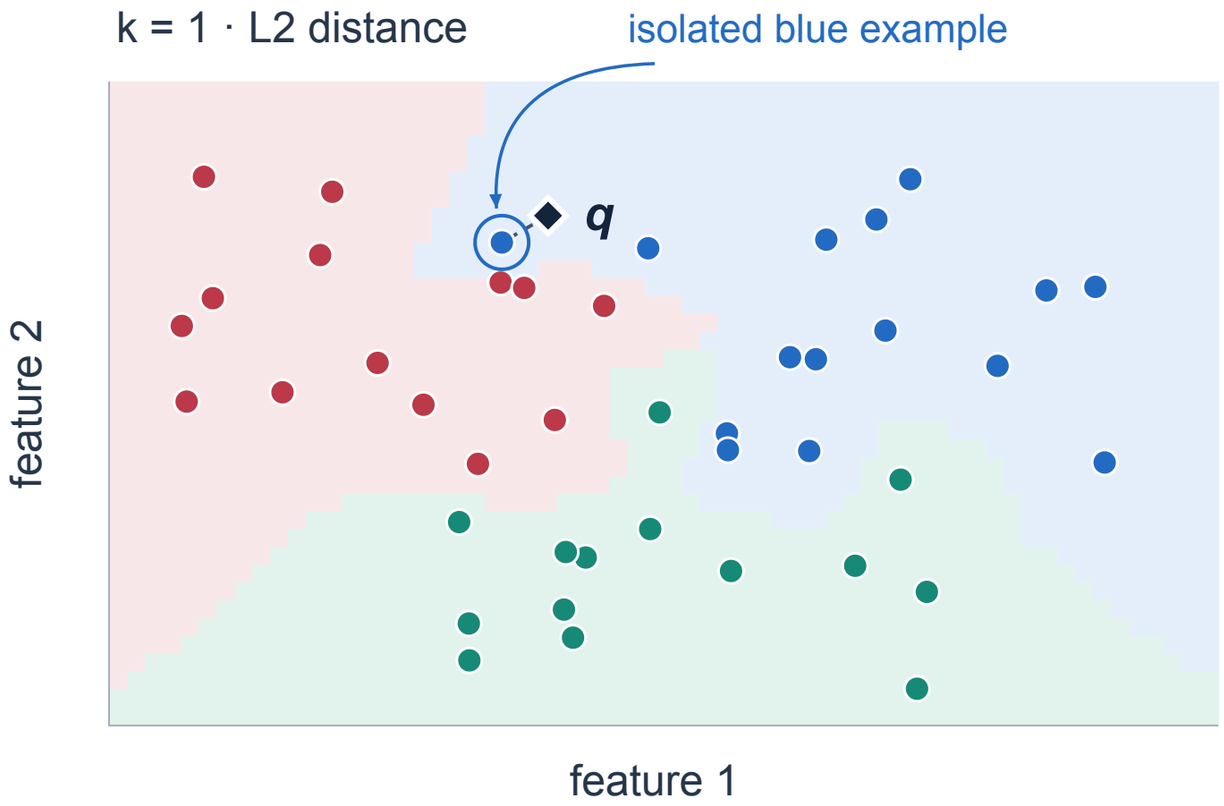
$$\hat{y}(\mathbf{q}) = \text{mode}\{y^{(i)} : i \in \mathcal{I}_k(\mathbf{q})\}$$

Choose the most frequent label among these k examples.

Larger neighborhoods can reduce sensitivity to individual examples.

For a vote tie, use the tied class whose nearest neighbor is closest.

How k changes the decision regions



Neighbors k: **1**

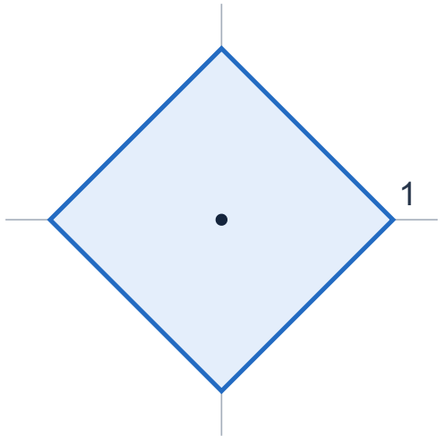
Prediction for q: blue

Background colors show predicted labels. Dots are stored examples.

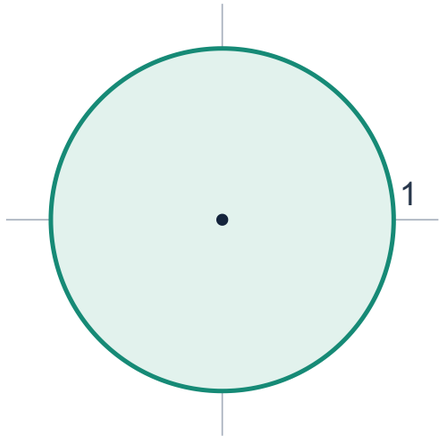
> Increase k from 1 to 3. Does $\hat{y}(\mathbf{q})$ stay blue?

Click the plot to move query $\mathbf{q}$.

L1 and L2 define different neighborhoods



L1: diamond



L2: circle

$$d_{\mathrm{L1}}(\mathbf{q}, \mathbf{x}^{(i)}) = \sum_{j=1}^{D} |q_j - x_j^{(i)}|$$

$$d_{\mathrm{L2}}(\mathbf{q}, \mathbf{x}^{(i)}) = \sqrt{\sum_{j=1}^{D} (q_j - x_j^{(i)})^2}$$

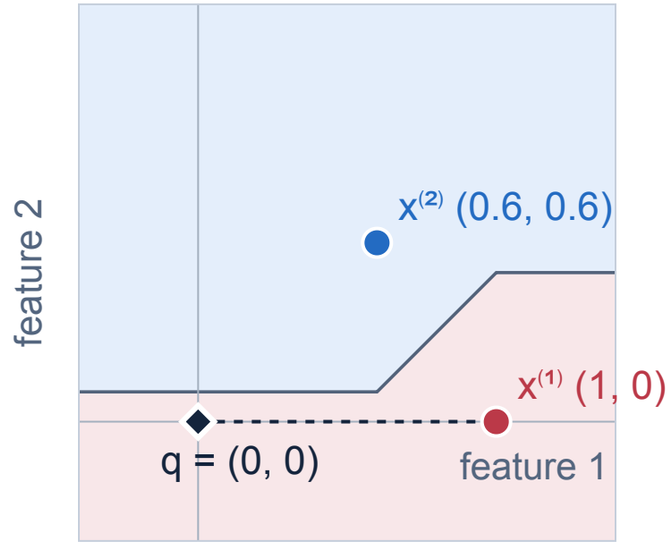
Contours show $d(\mathbf{q}, \mathbf{0}) = 1$ in two dimensions.

L2 preserves distances under orthogonal rotations. L1 generally does not.

The distance metric can change the boundary

L1 distance

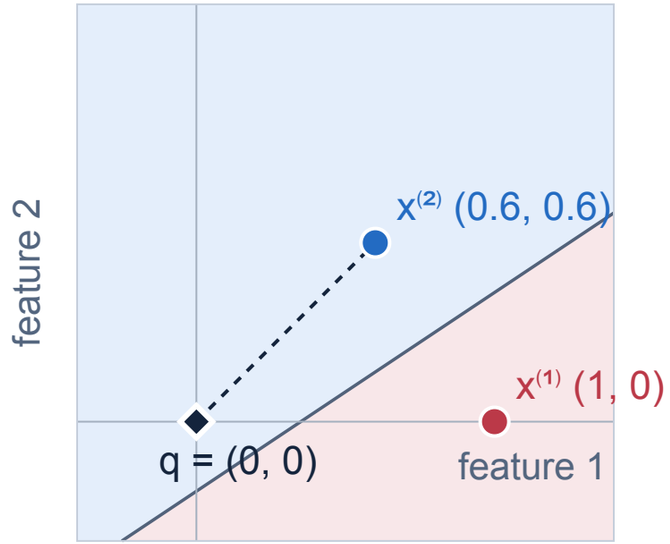
Prediction for q: class A



$d(q, x^{(1)}) = 1.00$ $d(q, x^{(2)}) = 1.20$

L2 distance

Prediction for q: class B



$d(q, x^{(1)}) = 1.00$ $d(q, x^{(2)}) = 0.85$

Fixed examples: $\mathbf{x}^{(1)}$ has class A; $\mathbf{x}^{(2)}$ has class B. Only the metric changes.

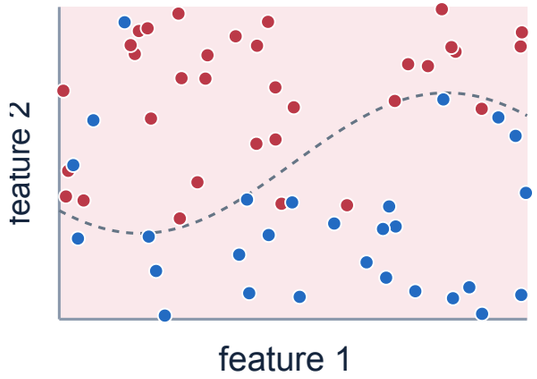
Model selection

How should we choose k and the distance metric?

Underfitting and overfitting

Generalization: predicting labels for new examples from the target distribution.

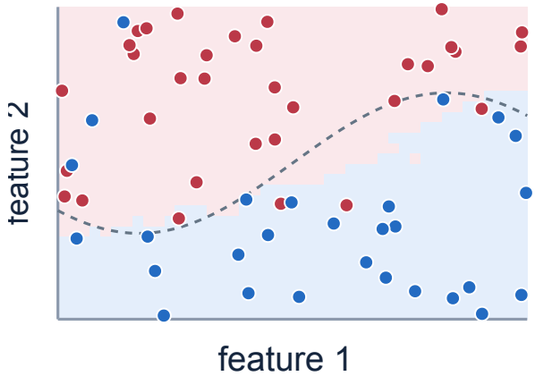
Underfitting · k = 61



Training error: 45.9%
Validation error: 50.0%

Too rigid to capture the pattern.

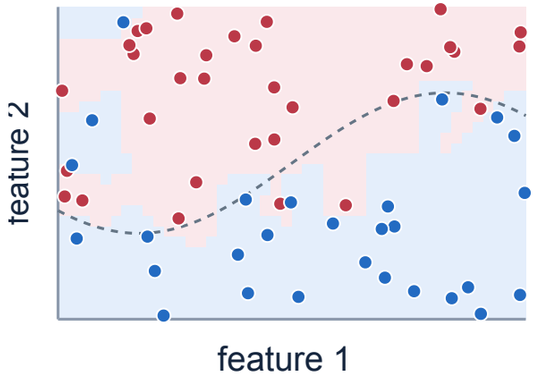
Useful fit · k = 5



Training error: 13.1%
Validation error: 6.4%

Captures the pattern despite noisy labels.

Overfitting · k = 1

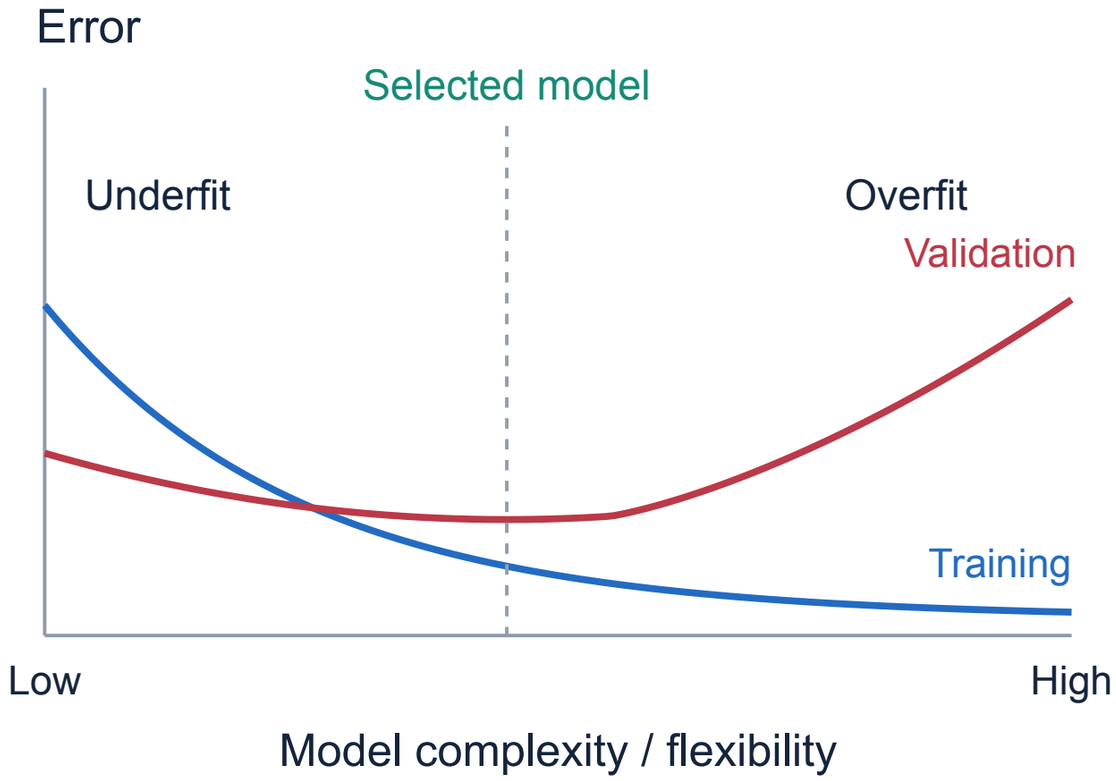


Training error: 0.0%
Validation error: 14.5%

Fits training noise; performs worse on new examples.

Dashed: underlying boundary. Synthetic example: 61 training points with 6 flipped labels; 3,721 clean validation points.

Model complexity and generalization



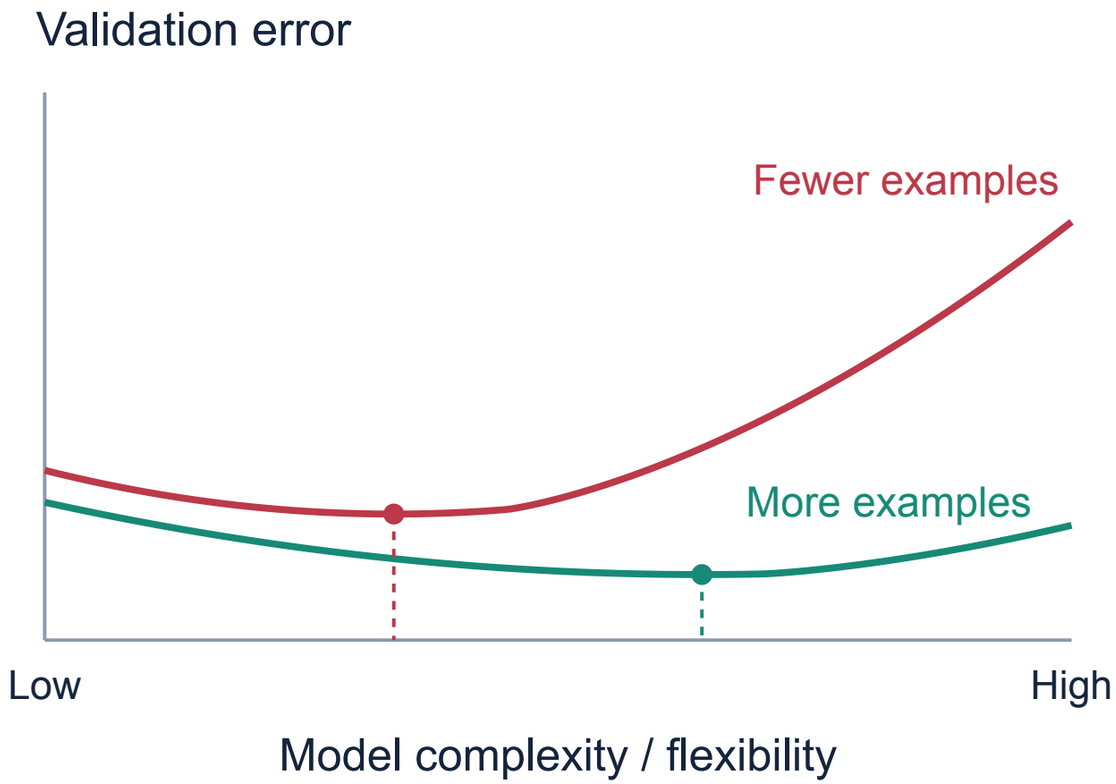
Schematic trend, not a universal curve.

Underfitting: the model cannot capture the pattern, or has not been fitted well.

Overfitting: fitting training-specific details harms predictions on unseen data.

> Select complexity using validation error, then evaluate once on the test set.

Training data and model complexity



Schematic: same task, comparable data quality and fitting procedure.

More representative training data can reduce overfitting.

A more flexible model may become useful when more data supports it.

> More parameters alone do not guarantee better generalization.

Which choices should we tune?

Neighborhood size

$$k \in \{1, 3, 5, \ldots\}$$

How much local evidence should contribute?

Distance metric

$$d \in \{d_1, d_2, \ldots\}$$

What makes two representations similar?

> Choose hyperparameters by measuring performance on held-out validation data.

Two tempting evaluation mistakes

Choose k using training accuracy?

With distinct stored inputs, 1-NN can classify each training example using itself.

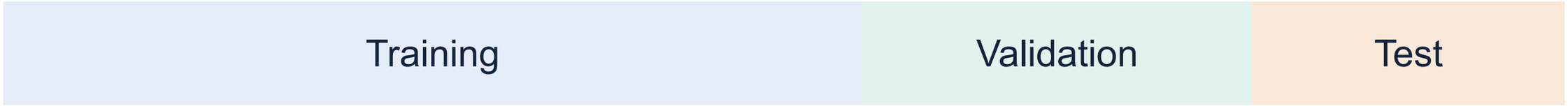
> This does not measure performance on unseen images.

Choose k using test accuracy?

The test set then influences the model choice.

> Its score no longer provides an independent final evaluation.

Training, validation, and test



Training	Fit parameters or store examples.
Validation	Compare choices such as k, metric, and representation.
Test	Evaluate the selected procedure after choices are fixed.

> For evaluation on new scenes, keep all views of each scene in one split to avoid information leakage.

Cross-validation reuses limited data



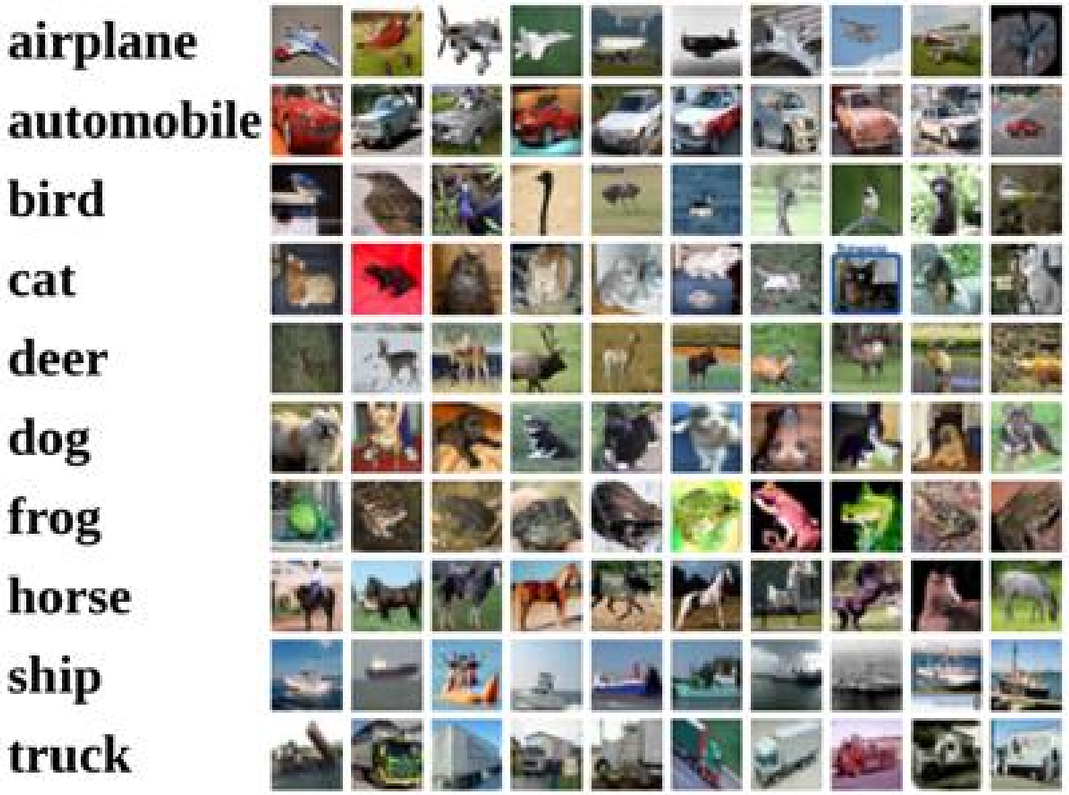
Fit a fresh model in every run. Average validation accuracy.

For each candidate hyperparameter setting:

1. Fit on four folds.
2. Validate on the remaining fold.
3. Rotate the held-out fold and average.

After selection, fit on all development data and evaluate on the untouched test set.

CIFAR-10 as a working example



10 categories

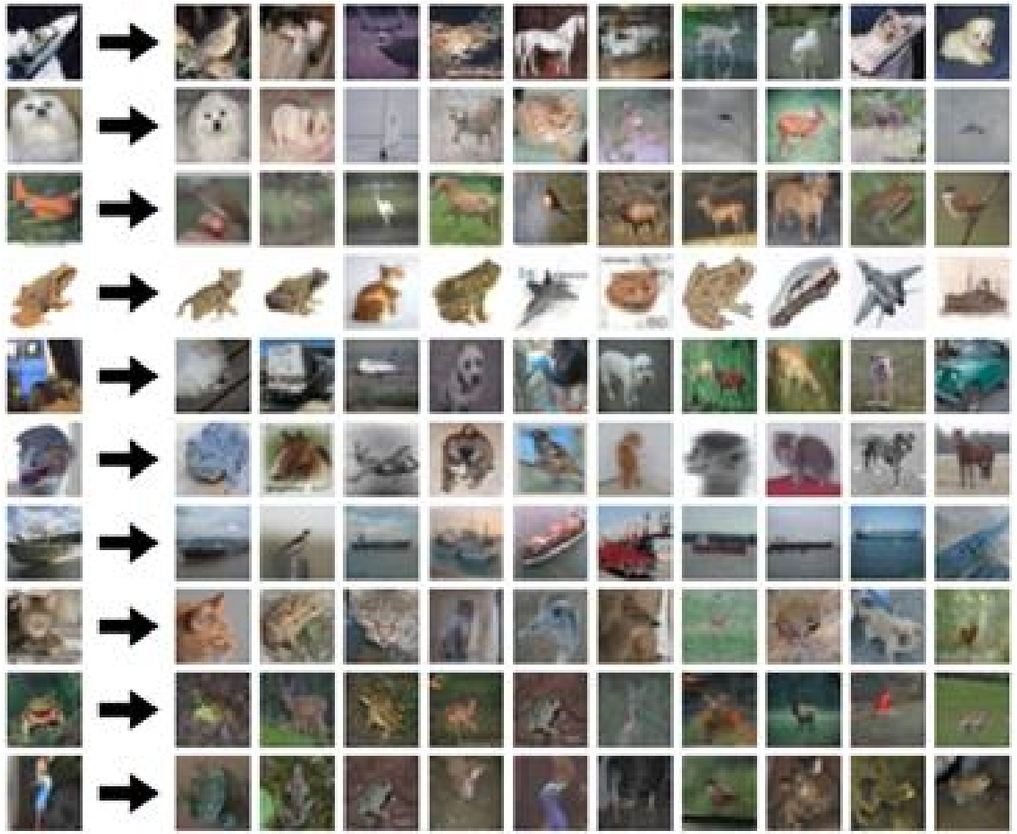
50,000 training images

10,000 test images

$$32 \times 32 \times 3 = 3072$$

Hold out validation data from the training portion.

Nearby pixels can retrieve the wrong class



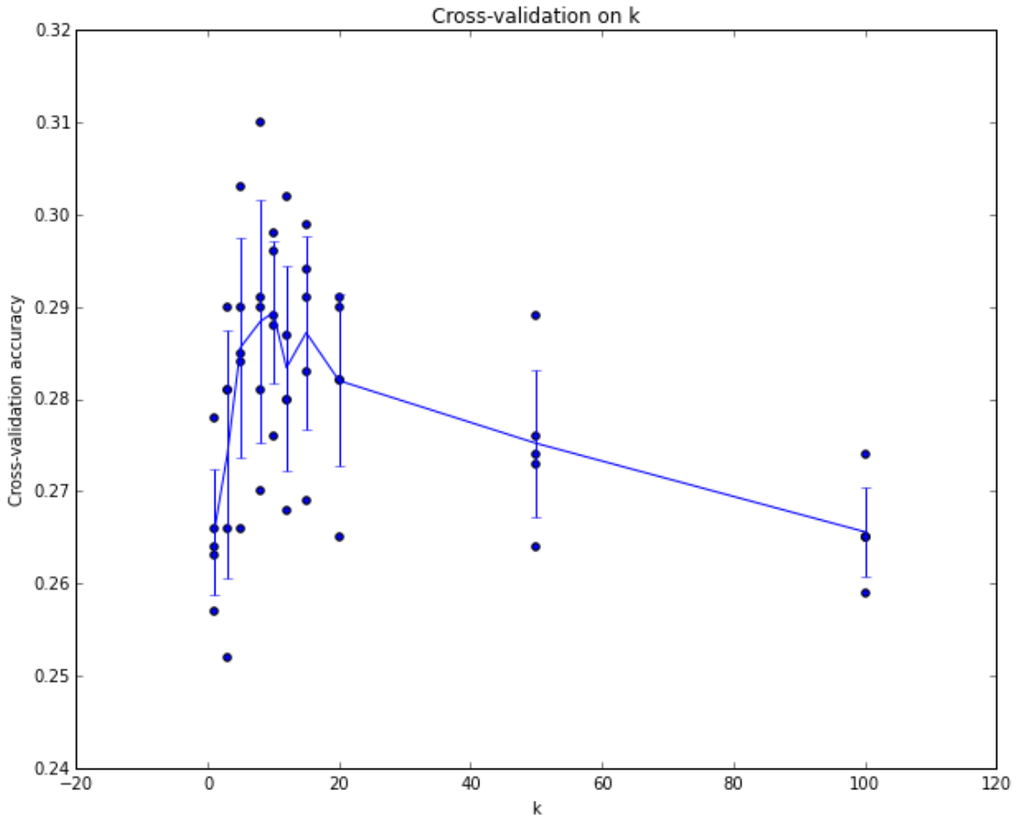
Leftmost image in each row: query.

Following images: nearest training images by pixel distance.

> Similar colors and backgrounds can dominate object identity.

Inspect a failure: what visual property makes the retrieved image close?

Selecting k from validation results



Each dot represents one fold.

The line joins fold means.

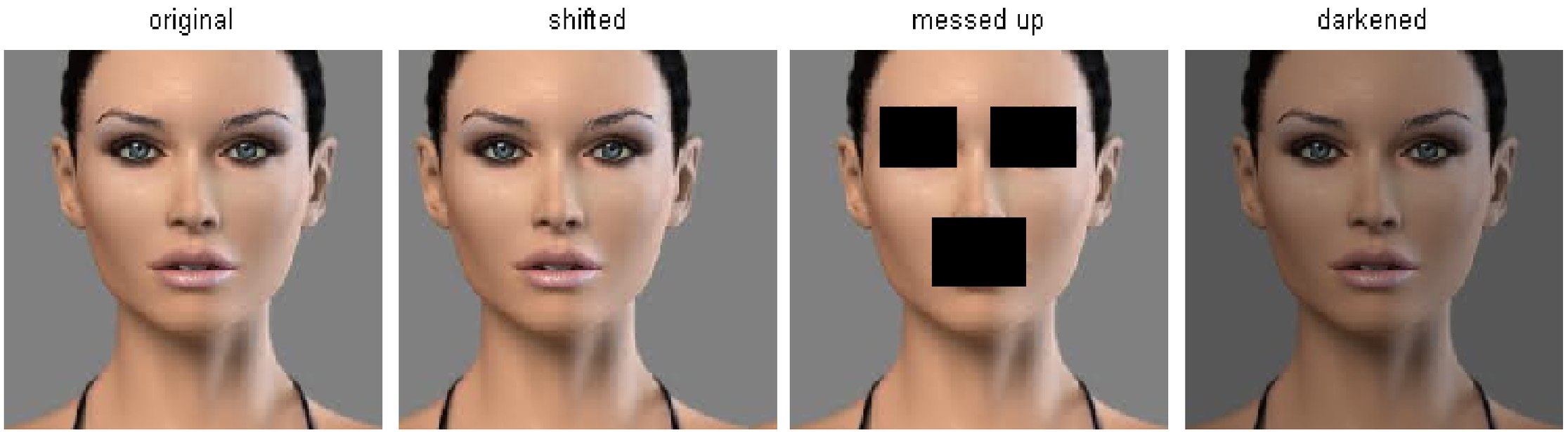
Bars show standard deviation across folds.

> In this example, mean validation accuracy peaks near $k = 7$.

Choose k using validation, then evaluate once on the test set.

Pixel distance and perceptual similarity

Each modified image has the same L2 distance from the original.



> Pixel distance does not reliably reflect perceptual similarity.

What nearest neighbors teach us

- The representation and distance determine which examples count as similar.
- k controls the size of the neighborhood used for a prediction.
- Validation supports model selection. Test data supports final evaluation.
- Raw pixel distance is a weak baseline for semantic recognition.

> Can we replace the stored dataset with a compact set of learned parameters?

Linear classification

Can learned parameters replace the stored training set?

$$s = Wx + b$$

A parametric score function

image x → $f(x; W, b)$ → C class scores

$$\hat{y} = \arg\max_c s_c(x)$$

Training

Estimate a fixed-size set of parameters.

Prediction

Compute scores using those parameters.

> Scores rank classes. They are not yet probabilities.

The linear classifier

$$\underbrace{s}_{C\times 1} = \underbrace{W}_{C\times D}\underbrace{x}_{D\times 1} + \underbrace{b}_{C\times 1}$$

$$s_c = w_c^\top x + b_c$$

Input	CIFAR-10 image flattened to $D = 3072$ values
Parameters	$W \in \mathbb{R}^{10\times 3072}, \quad b \in \mathbb{R}^{10}$
Output	Ten scores, one for each category

Strictly, the bias makes the mapping affine. “Linear classifier” is standard terminology.

Linear classifier: algebraic view

$$W = \begin{bmatrix} 0.2 & -0.5 & 0.1 & 2.0 \\ 1.5 & 1.3 & 2.1 & 0 \\ 0 & 0.25 & 0.2 & -0.3 \end{bmatrix} \qquad x = \begin{bmatrix} 56 \\ 231 \\ 24 \\ 2 \end{bmatrix}, \quad b = \begin{bmatrix} 1.1 \\ 3.2 \\ -1.2 \end{bmatrix}$$

The highlighted row contains the four weights used for the cat score.

$$Wx + b = \begin{bmatrix} -96.8 \\ 437.9 \\ 60.75 \end{bmatrix} \begin{matrix} \text{cat} \\ \text{dog} \\ \text{ship} \end{matrix}$$

> Next, place these same four weights back at their pixel locations.

Linear classifier: visual view

Image x

56	231
24	2

Keep the same pixel order

Cat row, reshaped

0.2	−0.5
0.1	2.0

$$s_{\text{cat}} = \langle w_{\text{cat}}, x \rangle + 1.1 = -96.8$$

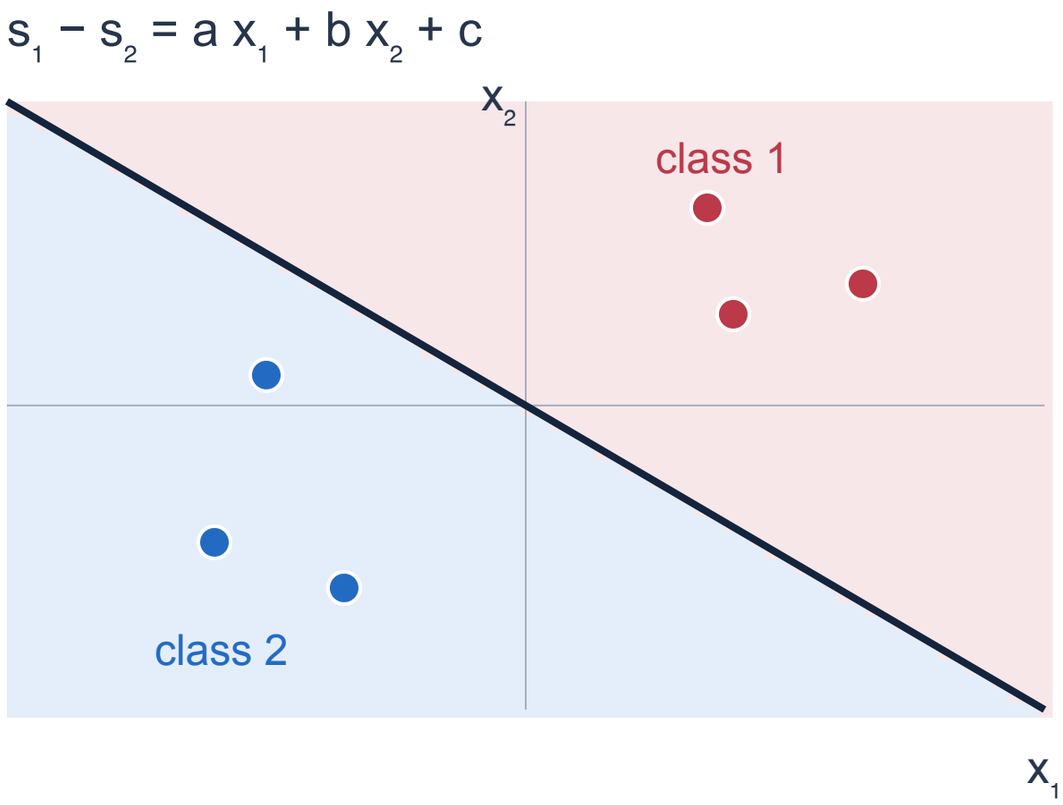
Multiply corresponding entries, sum, then add the bias.

For CIFAR-10, reshape each row to a 32 × 32 × 3 template



Colors show signed, rescaled weights. Each template combines varied appearances.

Two-class decision geometry



a **1.0**

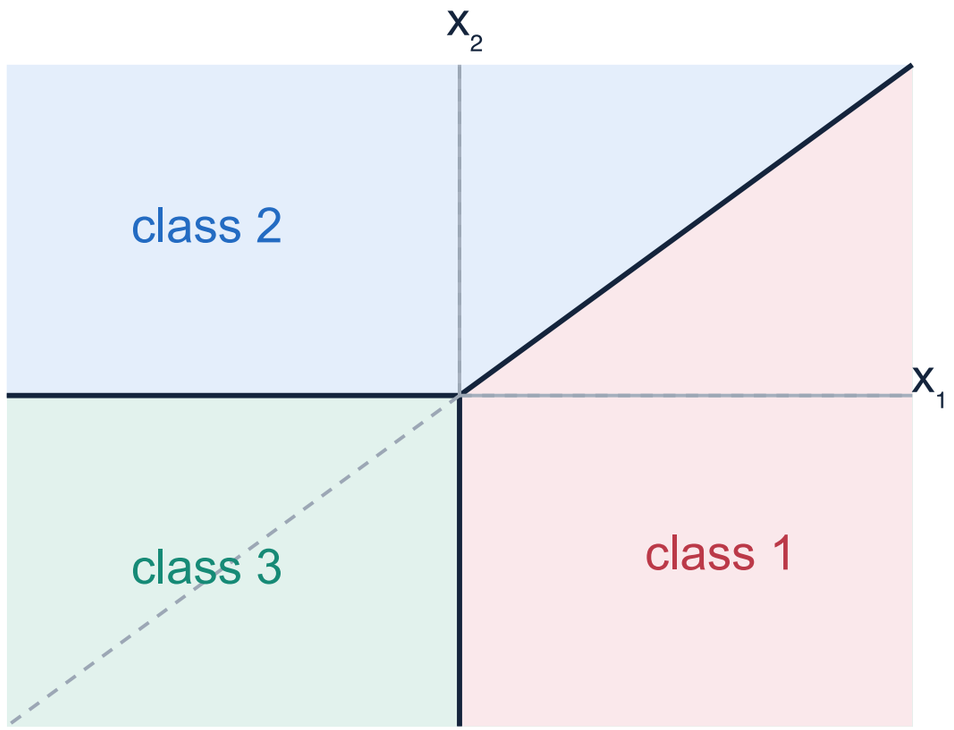
b **1.0**

c **0.0**

Two-class boundary: $s_1 = s_2$.
The normal is (a, b).

With more classes, compare every class score.

Multiclass decision regions



Solid: active class boundary

Dashed: equal scores, but another class wins

$$\hat{y} = \arg\max_c s_c(x)$$

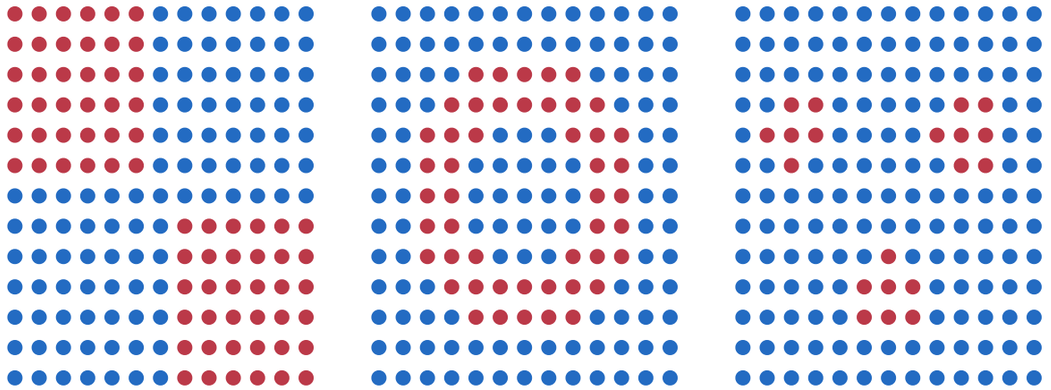
Example: $s_1 = x_1, \quad s_2 = x_2, \quad s_3 = 0.$

Class c wins where its score is at least every competing score.

$$(w_c - w_j)^T x + (b_c - b_j) \geq 0 \quad \forall j$$

> Each pairwise boundary is linear. A class region can have several boundary pieces.

Where a single linear boundary fails



> A two-class affine classifier cannot separate these labels with one hyperplane.

Nonlinear features can change what becomes linearly separable.

Nearest neighbors and linear classifiers

	Nearest neighbor / k-NN	Linear classifier
Training	Store examples; no weight fitting.	Optimize weights and biases.
Per-query cost	$O(ND)$ for direct distance search.	$O(CD)$ to compute all scores.
Prediction storage	$O(ND)$ training features.	$O(CD)$ model parameters.
Strengths	Simple baseline; flexible local regions; inspect retrieved neighbors.	Compact; fast scoring; interpretable class weights.
Limitations	Expensive search; depends strongly on representation and distance.	Restricted affine-score regions; may underfit the chosen features.

N examples, D features, C classes. Costs exclude feature extraction; approximate search can accelerate k-NN.

From scores to class probabilities

A score tells us which category wins.
Can a prediction also express how strongly the model favors each category?

Scores are not probabilities

Class	Score
cat	3.2
car	5.1
frog	−1.7

Prediction: car

A score is not a percentage.

Model probabilities:

$$p_c = P_\theta(Y = c \mid X = x)$$

$$p_c \geq 0, \qquad \sum_{c=1}^{C} p_c = 1$$

C classes; $\theta = (W, b)$.

Softmax probabilities

$$p_c = \mathrm{softmax}(s)_c = \frac{e^{s_c}}{\sum_{j=1}^{C} e^{s_j}}$$

Class	Logit	Positive weight	Probability
cat	3.2	24.533	0.1300
car	5.1	164.022	0.8690
frog	−1.7	0.183	0.0010

Exponentiate, then normalize.

Explore softmax probabilities



Probabilities sum to 1. Prediction: car.

Cat logit **3.2**

Car logit **5.1**

Frog logit **-1.7**

> What happens if we add the same constant to every logit?

Which prediction better supports the label?

Observed label: **cat**.

Model prediction	Cat	Car	Frog	Argmax
A	0.1300	0.8690	0.0010	car
B	0.3000	0.6000	0.1000	car

> Both predict car. Which assigns more probability to the observed cat label?

Maximize the likelihood of training labels

Training labels: $y_1, \ldots, y_N$; parameters: $\theta = (W, b)$.

$$\max_\theta \prod_i p_\theta(y_i \mid x_i)$$

$$\iff \quad \max_\theta \sum_i \log p_\theta(y_i \mid x_i)$$

$$\iff \quad \min_\theta \frac{1}{N} \sum_i \underbrace{-\log p_\theta(y_i \mid x_i)}_{\text{per-example loss}}$$

Assume labels are conditionally independent given inputs and parameters.

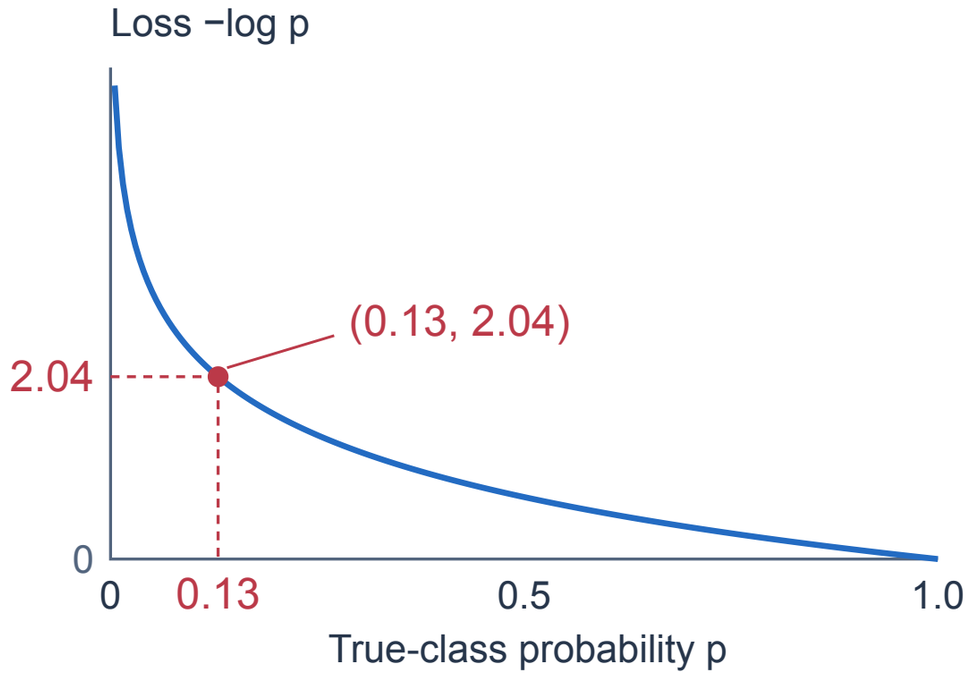
Cross-entropy loss

Observed label: cat

Logits: (3.2, 5.1, −1.7)

$$\ell = -\log p_y$$

$$p_{\text{cat}} \approx 0.1300 \quad \Rightarrow \quad \ell \approx 2.0404$$



Checking the behavior of softmax loss

If all C scores are equal?

$$p_c = 1/C, \qquad \ell = \log C$$

For ten classes: $\ell \approx 2.3026$.

How small or large can the loss be?

$$p_y \to 1 \Rightarrow \ell \to 0$$

$$p_y \to 0 \Rightarrow \ell \to +\infty$$

A numerically stable implementation

$$\ell_i = \log \sum_j e^{s_j - m} + m - s_{y_i}, \qquad m = \max_j s_j$$

```
# S: (N, C); y: integer class indices, shape (N,)
z = S - S.max(axis=1, keepdims=True)
log_Z = np.log(np.exp(z).sum(axis=1))
loss = (log_Z - z[np.arange(len(y)), y]).mean()
```

Subtracting a common constant leaves softmax probabilities unchanged.

Prefer the framework’s fused cross-entropy function in training code.

From image categories to a learning objective

Component	Question
Representation and distance	What makes examples similar?
Validation protocol	How do we choose among alternatives?
Scores $Wx + b$	How does the model rank categories?
Softmax probabilities	How is probability mass shared?
Cross-entropy	How well does the model explain observed labels?

Exit check

1. Why can a shifted image have the same label but a large pixel distance?

   Corresponding pixels change, while object identity can remain stable.

2. Which split chooses k? Which split reports the final result?

   Validation chooses k. The untouched test set evaluates the final procedure.

3. For two equal logits, what are the probabilities? With a known label, what is the loss?

   $p = (1/2, 1/2)$; $-\log(1/2) = \log 2 \approx 0.6931$.

Reading list

- **CS231n · Image classification**

  Nearest neighbors, distances, validation, and model selection.

- **CS231n · Linear classification**

  Class scores, softmax probabilities, and negative log likelihood.

- Foundations of Computer Vision · **Chapter 9** and **Chapter 11**

  §§9.2, 9.5, 9.7.3: learning and risk; §11.2: underfitting and overfitting.

- Supplementary · **Bag of visual words and spatial pyramids**

  Sivic & Zisserman, ICCV 2003: local descriptors → visual vocabulary → image histogram.
  Lazebnik et al., CVPR 2006: spatial pyramid matching preserves coarse layout.